المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵎⴰⵙⵙⴰⵏⵉⵏ ⵜⵓⵏⵉⵙⵉⵏⵉ - ⴱⵏⵉ ⵎⵍⴰⵍ

Ecole Nationale des Sciences Appliquées - Béni Mellal

Ecole nationale des sciences appliquées de Beni-Mellal

Niveau : 1ère année
Filière : IACS

TP4 : Architecture Orientée Services (SOA)

Réalisé par:
TOHANI Marwa

Encadré par:
Pr. BE ELBAGHAZAOUI

Objectif :

L’objectif de ce tp est de créer une application ecommerce simple utilisant Spring boot pour implementer une architecture SOA .

Architecture du Système :

Eureka-server : permet au microservices de s’enregistrer et trover mutuellement

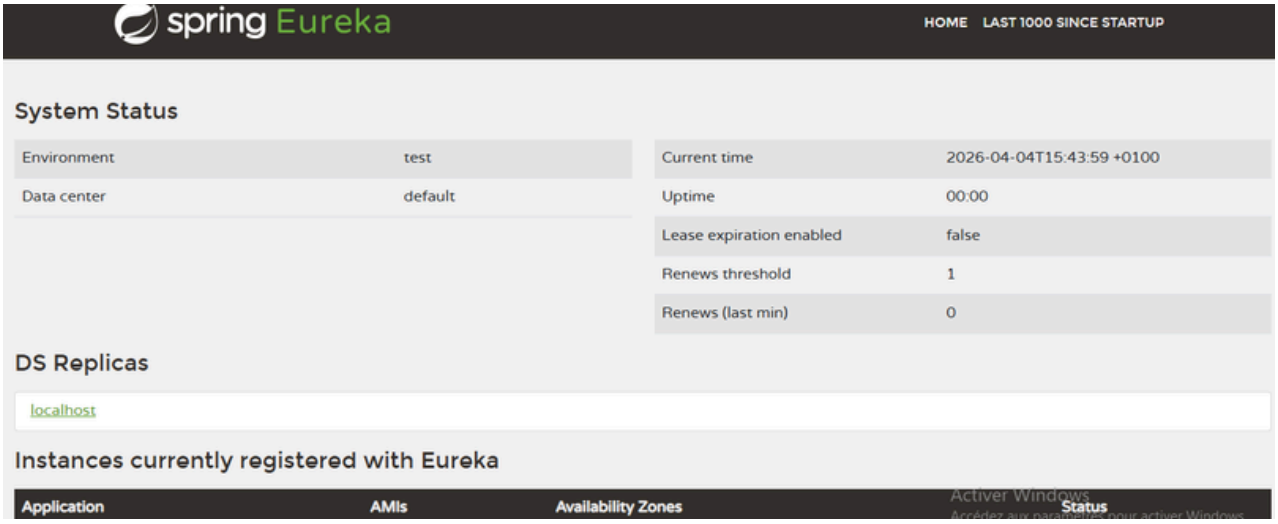
Product service : gestion des produits

Product order : gestion des commandes des clients

API Gateway : point d’entrée pour les requetes clients

Les étapes de réalisation :

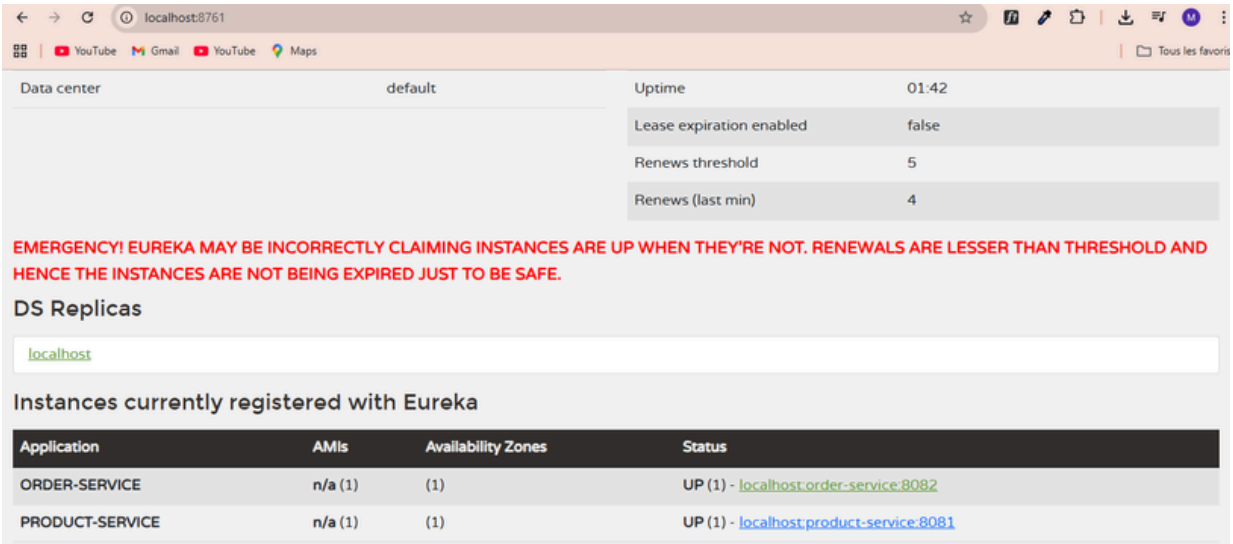
1. Création du service eureka-server qui utilise le port 8761



2. Création du service product (port 8081)
3. Activation de eureka dans la classe principale
4. Création de service order (port 8082)

Tester l'application :

- Enregistrement des services sur Eureak-server



- Test avec postman

Création d'un produit :

POST http://localhost:8081/api/products Send

Docs Params Authorization Headers (9) Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 { "name": "Laptop", "description":
2 "Ordinateur portable haute performance", "price": 1200.00, "stock": 10 }
```

Body Cookies Headers (5) Test Results (1/1) 200 OK 2.34 s 261 B Save Response

{} JSON Preview Visualize

```
1 {
2     "id": 1,
3     "name": "Laptop",
4     "description": "Ordinateur portable haute performance",
5     "price": 1200.0,
6     "stock": 10
7 }
```

Lister les produits:

GET http://localhost:8081/api/products Send

Docs Params Authorization Headers (7) Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL

This request does not have a body

Body Cookies Headers (5) Test Results (1/1) 200 OK 2.31 s 263 B Save Response

{} JSON Preview Visualize

```
1 [
2     {
3         "id": 1,
4         "name": "Laptop",
5         "description": "Ordinateur portable haute performance",
6         "price": 1200.0,
7         "stock": 10
8     }
9 ]
```

Créer une commande :

Créer une commande :

POST http://localhost:8082/api/orders?productId=1&quantity=2 Send

Docs Params • Authorization Headers (8) Body Scripts • Tests • Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL

This request does not have a body

Body Cookies Headers (5) Test Results (1/1) 200 OK 8.26 s 296 B Save Response

{} JSON Preview Visualize


```
{
    "id": 1,
    "productId": 1,
    "productName": "Laptop",
    "quantity": 2,
    "totalPrice": 2400.0,
    "orderDate": "2026-04-04T17:28:06.8847266",
    "status": "PENDING"
}
```


Activer Windows

Lister les commandes:

GET http://localhost:8082/api/orders Send

Docs Params Authorization Headers (7) Body Scripts • Tests • Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL

This request does not have a body

Body Cookies Headers (5) Test Results (1/1) 200 OK 717 ms 297 B Save Response

{} JSON Preview Visualize


```
[
    {
        "id": 1,
        "productId": 1,
        "productName": "Laptop",
        "quantity": 2,
        "totalPrice": 2400.0,
        "orderDate": "2026-04-04T17:28:06.884727",
        "status": "PENDING"
    }
]
```


Activer Windows

Exercices :

Exercice 1 : Ajouter une fonctionnalité

- Modifier le product service

```java
package com.ecommerce.product_service.model;

import ...
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id
    private String name;
    private String description;
    private Double price;
    private Integer stock;

    public void reduceStock(int quantity) {  no usages
        if (this.stock < quantity) {
            throw new RuntimeException("Stock insuffisant pour le produit :
        }
        this.stock -= quantity;
    }
}
```

- Modifier le order service

```java
public class OrderServiceApplication {

    public static void main(String[] args) { SpringApplication.run(OrderSer
    @Bean  new *
    @LoadBalanced // Très important pour utiliser le nom "product-service"
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

```java
package com.ecommerce.order_service.service;

import com.ecommerce.order_service.client.ProductClient;
import com.ecommerce.order_service.dto.ProductDTO;
import com.ecommerce.order_service.model.Order;
import com.ecommerce.order_service.repository.OrderRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;
import java.util.List;

@Service
@RequiredArgsConstructor
public class OrderService {

    private final OrderRepository orderRepository;
    private final ProductClient productClient;

    public List<Order> getAllOrders() {
        return orderRepository.findAll();
    }
```

```java
public class OrderService {
    public Order createOrder(Long productId, Integer quantity) {

        // 2. Étape 13 : Appeler l'endpoint PUT du Product Service pour déduire le stock
        // On suppose que productClient possède une méthode updateStock(id, quantity)
        // Si le stock est insuffisant, le Product Service renverra une erreur 400
        try {
            productClient.updateStock(productId, quantity);
        } catch (Exception e) {
            // Étape 14 : Gestion de l'erreur si le stock est insuffisant ou service injoignable
            throw new RuntimeException("Erreur lors de la validation du stock : " + e.getMessage());
        }

        // 3. Si la mise à jour du stock a réussi, on prépare la commande
        Order order = new Order();
        order.setProductId(productId);
        order.setProductName(product.getName());
        order.setQuantity(quantity);
        order.setTotalPrice(product.getPrice() * quantity);
        order.setOrderDate(LocalDateTime.now());
        order.setStatus("CREATED");

        // 4. Sauvegarder la commande en base de données
        return orderRepository.save(order);
```